DEVALIVE

API Architecture Document (AAD)

Version: 1.0

Prepared By: Tanmay Bonde

Architecture Type: RESTful API

Status: Approved

---

1. Introduction

Purpose

This document defines the complete API architecture for DevAlive.

It serves as the contract between:

- Frontend Team
- Backend Team
- QA Team
- AI Development Systems

The API architecture defines:

- Endpoints
- Request Formats
- Response Formats
- Authentication Rules
- Validation Rules
- Error Handling Standards
- Pagination Standards

---

2. API Design Principles

DevAlive follows:

REST Architecture

Resources are represented using URLs.

Example:

/api/projects
/api/logs
/api/notifications

JSON Communication

All requests and responses use JSON.

Stateless Requests

Every protected request must contain a valid JWT token.

Consistent Response Structure

Every API must return a standard format.

---

3. Base URL Structure

Development

http://localhost:5000/api

Production

https://api.devalive.com/api

---

4. Authentication Strategy

Authentication Method

JWT Authentication

Header Format

Authorization: Bearer <token>

Protected Routes Require:

Valid JWT Token

Public Routes:

Register
Login
Forgot Password
Reset Password

---

5. Standard Success Response

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {}
}
```

---

6. Standard Error Response

```
{
  "success": false,
  "message": "Error description",
  "errors": []
}
```

---

7. Authentication APIs

Register User

Endpoint

POST /auth/register

Access

Public

Request

```
{
  "fullName": "Tanmay Bonde",
  "email": "tanmay@gmail.com",
  "password": "Password123"
}
```

Success Response

```
{
  "success": true,
  "message": "Account created successfully"
}
```

Validation

- Full Name Required

- Email Required
- Unique Email
- Password Minimum 8 Characters

Login User

Endpoint

POST /auth/login

Access

Public

Request

```
{
  "email": "tanmay@gmail.com",
  "password": "Password123"
}
```

Success Response

```
{
  "success": true,
  "token": "jwt_token",
  "user": {}
}
```

Get Current User

Endpoint

GET /auth/me

Access

Protected

Response

```
{
  "success": true,
  "data": {
    "user": {}
  }
}
```

Logout User

Endpoint

POST /auth/logout

Access

Protected

---

8. Project APIs

Create Project

Endpoint

POST /projects

Access

Protected

Request

```
{
  "projectName": "Portfolio API",
  "endpointUrl": "https://example.com/health",
  "projectType": "api",
  "monitoringInterval": 15
}
```

Validation

- Project Name Required
- Valid URL Required
- Monitoring Interval Required

Response

```
{
  "success": true,
  "message": "Project created successfully"
}
```

Get All Projects

Endpoint

GET /projects

Access

Protected

Query Parameters

```
?page=1
&limit=10
&status=online
&search=portfolio
```

Response


```
{
  "success": true,
  "data": {
    "projects": [],
    "pagination": {}
  }
}
```

Get Single Project

Endpoint

GET /projects/:projectId

Access

Protected

Update Project

Endpoint

PUT /projects/:projectId

Access

Protected

Delete Project

Endpoint

DELETE /projects/:projectId

Access

Protected

Toggle Monitoring

Endpoint

PATCH /projects/:projectId/toggle-monitoring

Access

Protected

Purpose

Enable / Disable monitoring.

---

9. Monitoring APIs

Get Project Logs

Endpoint

GET /monitoring/logs

Access

Protected

Query Parameters

?page=1
&limit=20

&projectId=123
&status=success

Response


```
{
  "success": true,
  "data": {
    "logs": [],
    "pagination": {}
  }
}
```

Get Project Monitoring History

Endpoint

GET /monitoring/history/:projectId

Access

Protected

Purpose

Retrieve monitoring records for a specific project.

Trigger Manual Health Check

Endpoint

POST /monitoring/check/:projectId

Access

Protected

Purpose

Run immediate health check.

---

10. Analytics APIs

Dashboard Summary

Endpoint

GET /analytics/dashboard

Access

Protected

Response


```
{
  "success": true,
  "data": {
    "totalProjects": 10,
    "onlineProjects": 8,
    "offlineProjects": 2,
    "averageUptime": 99.2
  }
}
```

Project Analytics

Endpoint

GET /analytics/project/:projectId

Access

Protected

Response


```
{
  "success": true,
  "data": {
    "uptimePercentage": 99.5,
    "totalChecks": 500,
    "failedChecks": 3,
    "averageResponseTime": 280
  }
}
```

Uptime Chart Data

Endpoint

GET /analytics/uptime-chart/:projectId

Access

Protected

Purpose

Returns chart-ready data.

---

11. Notification APIs

Get Notifications

Endpoint

GET /notifications

Access

Protected

Query Parameters

?page=1
&limit=20

Mark Notification As Read

Endpoint

PATCH /notifications/:notificationId/read

Access

Protected

Delete Notification

Endpoint

DELETE /notifications/:notificationId

Access

Protected

---

12. User Settings APIs

Get Settings

Endpoint

GET /settings

Access

Protected

Update Settings

Endpoint

PUT /settings

Access

Protected

Request

```
{
  "emailNotifications": true,
  "downtimeAlerts": true,
  "recoveryAlerts": true,
  "defaultMonitoringInterval": 15
}
```

---

13. Pagination Standard

Request

?page=1
&limit=10

Response


```
{
  "pagination": {
    "currentPage": 1,
    "totalPages": 5,
    "totalRecords": 50,
    "hasNextPage": true,
    "hasPreviousPage": false
  }
}
```


---

14. Filtering Standard

Projects

?status=online
?status=offline

Logs

?status=success
?status=failure

Search

?search=portfolio

---

15. HTTP Status Codes

Success

200 OK
201 Created

Client Errors

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Validation Error

Server Errors

500 Internal Server Error
503 Service Unavailable

---

16. Security Standards

All Protected Routes Require JWT

Input Validation Required

- Email Validation
- URL Validation
- ObjectId Validation

Rate Limiting Applied To

Login
Register
Manual Health Check

Request Sanitization Required

Prevent:

- XSS
- NoSQL Injection
- Malicious Payloads

---

17. API Versioning Strategy

Current Version

/api/v1

Example

/api/v1/auth/login
/api/v1/projects
/api/v1/analytics/dashboard

Future Versions

/api/v2

---

18. API Module Summary

Authentication Module

/auth/*

Projects Module

/projects/*

Monitoring Module

/monitoring/*

Analytics Module

/analytics/*

Notifications Module

/notifications/*

Settings Module

/settings/*

---

19. API Architecture Summary

The DevAlive API Architecture follows a secure, scalable, RESTful design that provides:

- Authentication Management
- Project Management
- Monitoring Operations
- Analytics Processing
- Notification Management
- User Preferences

The API structure is designed to allow frontend, backend, testing, and AI development teams to work independently while maintaining complete integration compatibility.